

Phase 1 – Source Control & Git Governance Foundation

Objective

Establish a controlled Git governance model for cube CI/CD implementation using separate project and framework repositories.

Current Implementation

Repositories

wealth-cube

Purpose: delivery repository for SSAS cube model and environment config.

Current artifact contents:

```
cube-artifact/  
├── config/  
│   ├── datasources/  
│   └── roles/  
└── model/  
    └── Wealth.bim
```

wealth-devops-framework

Purpose: centralized deployment framework.

Current script source:

```
wealth-devops-framework/  
└── scripts/  
    └── cube/  
        ├── deploy-cube.ps1  
        ├── update-datasource.ps1  
        ├── update-roles.ps1  
        └── validate-model.ps1
```

Branching Strategy

Branch	Purpose	Environment
feature/*	active development and DEV testing	DEV
develop	integration branch for UAT	UAT
main	production-approved code	PROD
hotfix/*	emergency production fix	PROD, then sync back

Controls

- Direct push blocked to develop and main
- Pull request required for protected branches
- PR validation pipeline attached as build validation
- Squash merge preferred
- Work item linking required
- PR comments must be resolved
- Force push disabled
- Branch naming convention enforced

Deliverables

- ✓ wealth-cube repo structured
- ✓ wealth-devops-framework repo structured
- ✓ GitFlow branch model defined
- ✓ Branch-to-environment mapping defined
- ✓ PR governance defined
- ✓ Centralized framework repo introduced

Pending / To Review

- ☐ Confirm AD groups for approvals
- ☐ Confirm branch security for framework repo
- ☐ Confirm hotfix approval flow with client

Phase 2 – Infrastructure & Access Setup

Objective

Prepare self-hosted agent, service accounts, tools, and connectivity required for SSAS cube deployment.

Current Implementation

Self-hosted Agent

Agent machine:

DWWS01AD0A001.wintrust.wtfc

Agent service account:

WINTRUST\SVC-WT-DEV-PBI-AzAgt

Agent service:

Azure Pipelines Agent

Status:

Running

Required Tools

Tool	Purpose	Status
Windows PowerShell	Run release deployment scripts	Required
PowerShell 7	Available, but not preferred for SSAS deployment	Installed
SSMS / TOM DLLs	Required for Analysis Services deployment	Available
Git	Repo checkout	Required
SqlServer module	Optional, not primary deployment path	Not relied upon

Microsoft.AnalysisServices
DLLs

TOM/TMSL deployment

Used

SSAS Connectivity

Current DEV SSAS server:

DWDVS01PBDA004.wintrust.wtfc

Port: 2383

Planned UAT:

UWDVS01PBDA004.wintrust.wtfc

Port: 2383

Planned PROD:

PWDVS01PBDA004.wintrust.wtfc

Port: 2383

Datasource Servers

DEV → DWDA001DMDDL1

UAT → UWDA001DMDDL1

PROD → PWDA001DMDDL1

Access Requirements

SSAS

Permission	Purpose
Server Admin / deployment-level access	Create/update SSAS database
Database process permission	Process deployed cube
Role update permission	Sync cube roles

SQL Server

Permission	Purpose
------------	---------

db_datareader	Cube processing reads source data
---------------	-----------------------------------

Active Directory

Permission	Purpose
Read AD groups	Resolve domain groups used inside cube roles

Deliverables

- ✓ Self-hosted agent configured
- ✓ Agent running with service account
- ✓ TOM/SSAS DLL loading approach validated
- ✓ SSAS connectivity from pipeline validated
- ✓ DEV SSAS deployment tested

Pending

- ☐ Confirm UAT SSAS access
- ☐ Confirm PROD SSAS access
- ☐ Confirm source DB read permission for UAT/PROD
- ☐ Confirm AD group access for cube role resolution
- ☐ Confirm final backup/shared path approach with client

Phase 3 – Continuous Integration / Cube Build Pipeline

Objective

Create a YAML-based build pipeline that validates and packages cube deployment artifacts.

Current Implementation

Build pipeline:

Wealth-Cube-Build

Pipeline file:

wealth-cube/pipelines/cube-build.yml

Template source:

wealth-devops-framework/templates/cube/build-cube.yml

Build Trigger Strategy

Build runs manually for now.

Branch usage:

Branch	Purpose
feature/*	create DEV deployable artifact
develop	create UAT deployable artifact
main	create PROD deployable artifact

Artifact Output

Published artifact:

cube-artifact

Contains only:

config/
model/

Scripts are intentionally excluded from cube artifact.

Why Scripts Are Excluded

Scripts are pulled separately in Classic Release from:

wealth-devops-framework

This keeps deployment framework centralized and reusable.

Current Artifact Structure

```
cube-artifact/  
├── config/  
│   ├── datasources/  
│   │   ├── dev.yml  
│   │   ├── uat.yml  
│   │   └── prod.yml  
│   └── roles/  
│       ├── dev.yml  
│       ├── uat.yml  
│       └── prod.yml  
└── model/  
    └── Wealth.bim
```

Validation Checks

Build pipeline should validate:

- ✓ Wealth.bim exists
- ✓ config/datasources/*.yml exists
- ✓ config/roles/*.yml exists
- ✓ artifact structure is correct
- ✓ no deployment scripts included in cube artifact

Deliverables

- ✓ YAML build pipeline implemented
- ✓ Central build template consumed

- ✓ Cube artifact generated
- ✓ Artifact contains model + config only
- ✓ Build supports branch-based artifact creation

Pending Improvements

- Add stricter validation for datasource config schema
- Add role config schema validation
- Add artifact metadata/version file
- Add build tagging for DEV/UAT/PROD traceability

Phase 4 – Classic Release Pipeline: Cube Deployment

Objective

Deploy cube artifact to SSAS DEV/UAT/PROD using Classic Release Pipeline with controlled approvals and branch-based deployment mapping.

Current Release Pipeline

Wealth-Cube-Release

Artifacts:

1. Wealth-Cube-Build
Source: Build artifact
Contains: cube-artifact
2. wealth-devops-framework
Source: Azure Repos Git
Branch: main
Version: latest from default branch
Contains: deployment scripts

Deployment Stages

DEV
UAT
PROD

Each stage has:

1 Agent job
1 PowerShell task

Task:

Deploy SSAS Cube (<ENV>)

Script path:

```
$(System.DefaultWorkingDirectory)/wealth-devops-  
framework/scripts/cube/deploy-cube.ps1
```

Arguments:

```
-BimPath "$(System.DefaultWorkingDirectory)/Wealth-Cube-Build/cube-  
artifact/model/Wealth.bim" -Environment "$(Environment)" -SsasServer  
"$(SsasServer)" -DatabaseName "$(DatabaseName)" -DatasourcesConfigFile  
"$(System.DefaultWorkingDirectory)/Wealth-Cube-Build/cube-  
artifact/$(DatasourceConfigPath)" -RolesConfigFile  
"$(System.DefaultWorkingDirectory)/Wealth-Cube-Build/cube-  
artifact/$(RolesConfigPath)" -ProcessType "$(ProcessType)" -  
CreateDatabaseIfNotExists "$(CreateDatabaseIfNotExists)"
```

Stage Trigger Mapping

Stage	Artifact Branch Filter	Approval
DEV	feature/*	No approval / testing approval
UAT	develop	Required
PROD	main	Required

Current Behavior

feature/* build → release created → DEV deploys
develop build → release created → UAT waits/deploys
main build → release created → PROD waits/deploys

Variable Groups

Linked variable groups:

Wealth-Common → DEV, UAT, PROD
Wealth-DEV → DEV
Wealth-UAT → UAT
Wealth-PROD → PROD

Current Variable Groups

Wealth-Common

MaxParallelConnection = 4
TimeoutSeconds = 600
TomDllPath = SSMS / TOM DLL path

Wealth-DEV

Environment = DEV
SsasServer = DWDVS01PBDA004.wintrust.wtfc
DatabaseName = Wealth_DEV_CICD
DatasourceConfigPath = config\datasources\dev.yml
RolesConfigPath = config\roles\dev.yml
modelName = Wealth
ModelPath = src\model\Wealth.bim
ProcessType = Full
CreateDatabaseIfNotExists = true
strictMode = false

Wealth-UAT

Currently placeholder values. Final values pending client access.

Expected:

Environment	= UAT
SsasServer	= UWDVS01PBDA004.wintrust.wtfc
DatabaseName	= <UAT database name>
DatasourceConfigPath	= config\datasources\uat.yml
RolesConfigPath	= config\roles\uat.yml
ProcessType	= Full / Default
CreateDatabaseIfNotExists	= true
strictMode	= false/true

Wealth-PROD

Currently placeholder values. Final values pending client access.

Expected:

Environment	= PROD
SsasServer	= PWDVS01PBDA004.wintrust.wtfc
DatabaseName	= <PROD database name>
DatasourceConfigPath	= config\datasources\prod.yml
RolesConfigPath	= config\roles\prod.yml
ProcessType	= Controlled
CreateDatabaseIfNotExists	= true/false based on client decision
strictMode	= true
BackupBeforeDeploy	= true

Correction needed:

Wealth-PROD description should be: Purpose: PROD environment variables

Deployment Logic

Deployment script performs:

1. Load Analysis Services libraries
2. Validate input paths

3. Create working BIM copy
4. Apply datasource config
5. Apply role config
6. Deserialize updated BIM
7. Connect to SSAS server
8. Deploy database using TMSL createOrReplace
9. Process model based on ProcessType
10. Log deployment result

Deliverables

- ✓ Classic Release pipeline created
- ✓ DEV/UAT/PROD stages created
- ✓ Cube build artifact linked
- ✓ Framework repo linked as second artifact
- ✓ Release stage branch filters configured
- ✓ PowerShell task parameterized
- ✓ Variable groups linked by environment
- ✓ DEV deployment tested successfully

Pending

- Validate UAT deployment with real UAT server
- Validate PROD deployment with real PROD server
- Finalize approver groups
- Decide framework repo versioning/tagging before PROD
- Decide final backup path/backup strategy

Phase 5 – Testing & Quality Controls

Objective

Ensure the deployed cube is validated before promotion to higher environments.

DEV Validation

After feature branch deployment to DEV:

- ✓ Deployment completes successfully
- ✓ Database created/updated on DEV SSAS
- ✓ Datasource config points to DEV source
- ✓ Roles applied from dev.yml
- ✓ Cube can be browsed
- ✓ Processing completes or is intentionally skipped

UAT Validation

After merge to develop:

- UAT release created from develop artifact
- UAT approval completed
- UAT SSAS deployment successful
- UAT datasource config applied
- UAT roles applied
- Business validation completed
- Defects linked to Azure DevOps work items

PROD Validation

After merge to main:

- PROD release created from main artifact
- PROD approval completed
- PROD deployment successful
- PROD datasource config applied
- PROD role config applied
- Smoke test completed
- Business signoff captured

Quality Controls

- ✓ PR validation before merge
- ✓ Branch-based release filtering
- ✓ Environment-specific variable groups
- ✓ Environment-specific config files
- ✓ Approval gates for UAT/PROD
- ✓ Deployment logs captured in release history

Recommended Additional Checks

- ❑ Validate BIM JSON before artifact publish
- ❑ Validate YAML config syntax
- ❑ Validate datasource names in config match BIM datasource names
- ❑ Validate role names in config match BIM role names
- ❑ Add pre-deployment dry-run / WhatIf mode
- ❑ Add post-deployment SSAS database existence check
- ❑ Add processing result validation

Phase 6 – Rollback & Operational Readiness

Objective

Provide a practical rollback process using previous successful release artifacts.

Current Rollback Strategy

Client decision:

No secondary deployment model

No VM backup dependency

Rollback will rely on redeploying previous successful release/artifact

Rollback Process

1. Identify last successful release
2. Open release history
3. Select previous successful release
4. Redeploy same stage
5. Validate SSAS database after deployment
6. Confirm processing state
7. Inform stakeholders

Important Current Risk

Framework repo is currently configured as:

```
main
latest from default branch
```

This means older cube artifact can be redeployed with newer deployment script.

Recommendation Before PROD

Use versioned framework script source:

```
DEV → latest from main acceptable during testing
UAT → fixed tag recommended
PROD → fixed framework tag required
```

Example:

```
wealth-devops-framework tag: v1.0.0
```

Deployment Logging

Current logs include:

```
Deployment started
Library loading
Configuration validation
Datasource update
```

Role update
SSAS connection
Database deployment
Processing result
Deployment status

Operational Readiness Checklist

- ☐ Document rollback steps
- ☐ Document known failure scenarios
- ☐ Document support contact
- ☐ Document required permissions
- ☐ Confirm release retention policy
- ☐ Confirm artifact retention policy
- ☐ Confirm framework script versioning approach
- ☐ Confirm notification process for failure

Phase 7 – Power BI Report CI/CD Future Plan

Status

Not implemented yet.

Power BI Report CI/CD should remain as a future phase and should not be shown as completed.

Objective

Implement CI/CD for Power BI reports after cube CI/CD foundation is stable.

Planned Scope

Reports will be migrated from:

PBIX

to:

PBIP

Planned Repository

Expected repo:

wealth-report

Multiple reports exist inside this repo.

Planned Build Pipeline

Future report build pipeline should:

1. Validate PBIP folder structure
2. Detect changed reports
3. Package report artifacts
4. Publish report artifact
5. Maintain environment-neutral report source

Planned Release Pipeline

Report deployment should target:

Power BI Workspace DEV

Power BI Workspace UAT

Power BI Workspace PROD

Planned Authentication

Use:

Service Principal

Power BI REST APIs

Planned Deployment Controls

feature/* → DEV workspace
develop → UAT workspace
main → PROD workspace

Planned Checks

- ☐ Workspace exists
- ☐ Service principal has access
- ☐ Dataset/report ownership handled
- ☐ Parameters updated per environment
- ☐ Refresh configured
- ☐ Gateway/data source mapping handled
- ☐ UAT/PROD approvals configured

Deliverables

- ☐ PBIP repo structure finalized
- ☐ Report build pipeline created
- ☐ Report release pipeline created
- ☐ Power BI service principal configured
- ☐ Workspace access configured
- ☐ Parameter rules implemented
- ☐ Report rollback strategy defined

Final Current Status Summary

Completed:

- ✓ Cube repo structure
- ✓ Framework repo structure
- ✓ YAML cube build pipeline
- ✓ Cube artifact generation
- ✓ Classic Release pipeline
- ✓ DEV/UAT/PROD stages

- ✓ Framework repo pulled separately in release
- ✓ Variable groups created and linked
- ✓ DEV deployment tested
- ✓ Branch filters configured for DEV/UAT/PROD
- ✓ UAT/PROD approvals configured initially

Pending:

- Actual UAT server/database access
- Actual PROD server/database access
- Final approver groups
- Framework repo versioning for PROD safety
- Rollback documentation
- Backup strategy confirmation
- Power BI Report CI/CD implementation